ADP32F03X 数字信号处理器

无源晶体配置应用说明

Advancechip Electronics

编号：JXDZ7.381.013 YYSM-Crystal

湖南进芯电子科技有限公司

2022 年 10 月

V1.0

目录

历史版本记录 2
1. ADP32F03X 无源晶体配置概述 4
2. 上电时序 5
3. 上电程序初始化流程图 7
4. 时钟源配置流程图 8
5. 时钟选择配置注意事项 9
6. 时钟切换异常导致影响 10

1. ADP32F03X 无源晶体配置概述

ADP32F03X 系列芯片支持内外部时钟源选择，GPIO38 或 GPIO19 管脚为有源时钟输入，X1、X2 管脚为无源时钟输入（片内集成振动器 OSC），OSC1、OSC2 为内部时钟源，芯片默认选择内部 OSC1 作为系统时钟输入源（如下图 1-1 黄线路径），外部时钟选择（如下图 1-1 红色路径）需用户通过程序配置完成，在外部无源时钟选择配置过程，需按照一定流程及时序进行初始化，否则芯片程序无法正常运行。

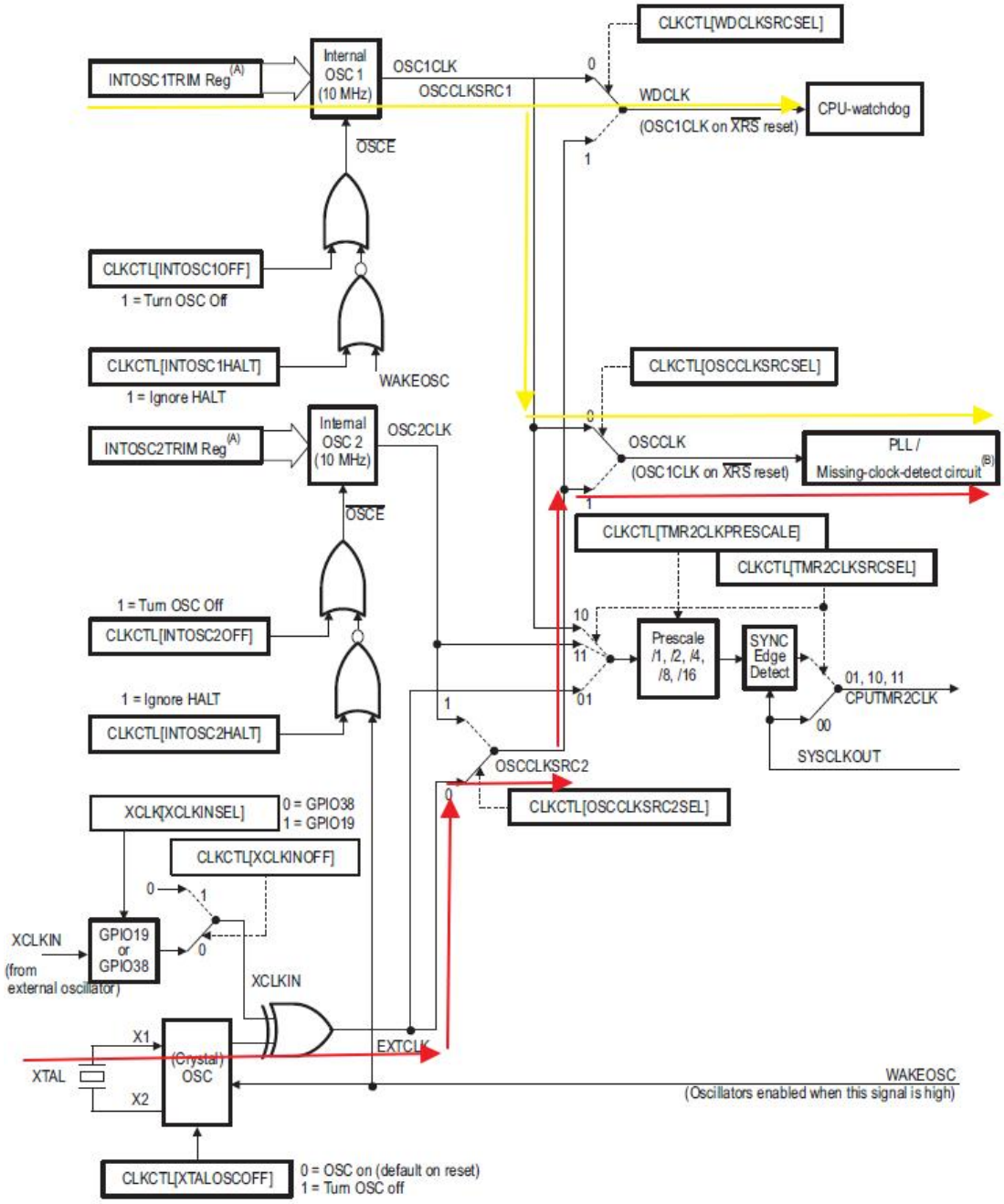


图 1-1

2. 上电时序

内部振动器 OSC 默认使能（如上图 1-1），所以无源晶体在上电过程已开始工作，用户在切换外部无源时钟前，需确保外部晶体正常工作（信号幅值到达 VDD/1.8 V *0.7），如下图为 VDDIO,XRS,X2 上电时序图，黄色为 VDDIO/3.3V, 红色的为 XRS，蓝色的 X2 引脚。

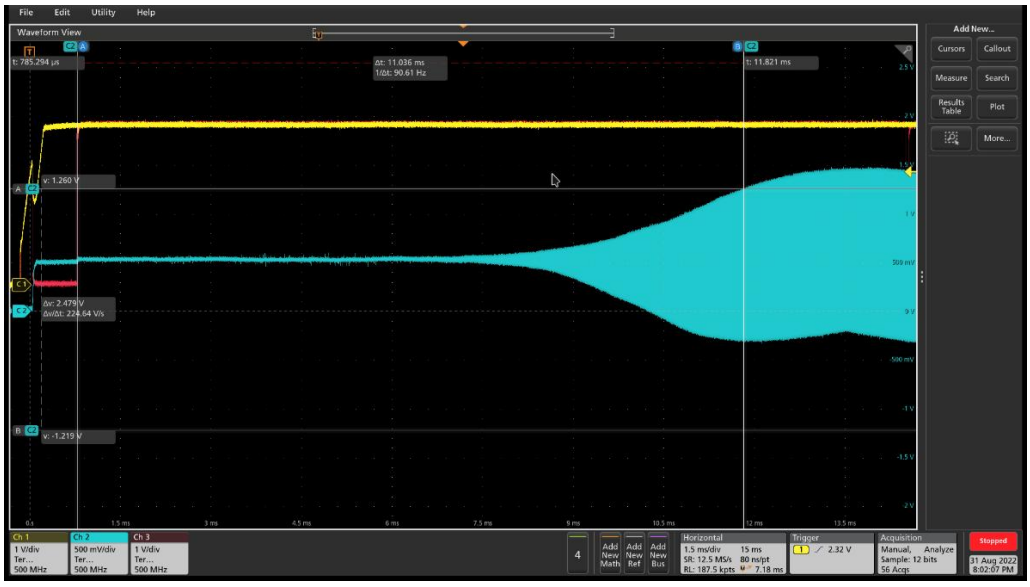
测试条件：单电源供电 VDDIO/3.3V;

复位引脚悬空（内部 POR 完成上电复位）；

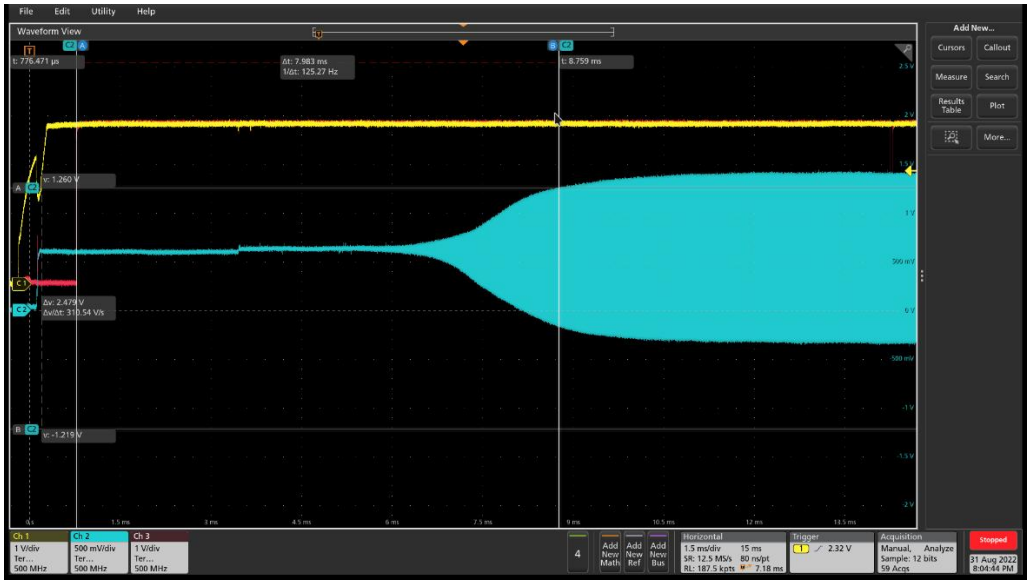
X1,X2 外接 10MHz 无源晶体,电容 22pF;

三温，-55℃，25℃，125℃；

1. 低温-55℃波形图，复位引脚拉高到外部晶体正常工作约 11ms；



2. 高温 125℃波形图，复位引脚拉高到外部晶体正常工作约 8ms；

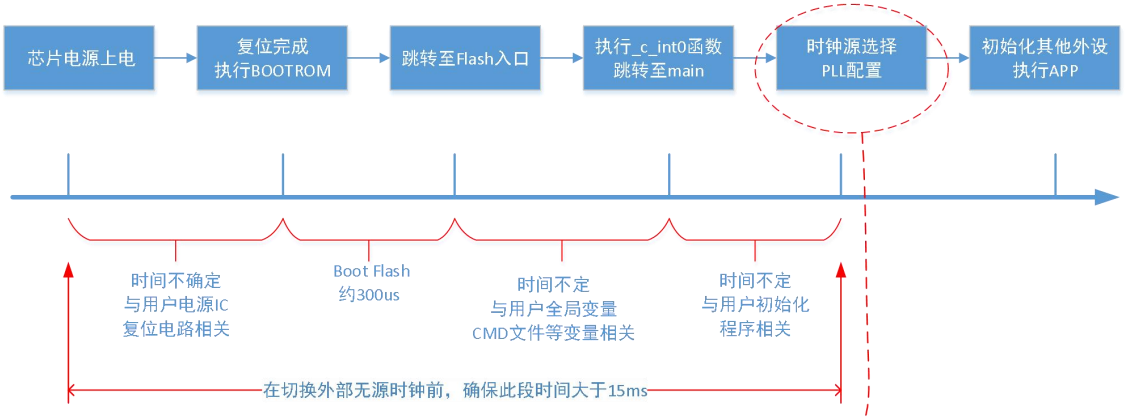


3. 常温 25℃波形图，复位引脚拉高到外部晶体正常工作约 9.6ms；



3. 上电程序初始化流程图

在切换外部时钟前，由于用户软硬件差异化（如下图 3-1 所示），导致系统初始化时间不确定，所以建议用户在程序切时钟源前等待时间大于 15ms,以确保程序正常运行。




```
void XtalOscSel (void)
{
    EALLOW;
    SysCtrlRegs.CLKCTL.bit.XTALOSCOFF = 0;     // 使能内部振动器（默认使能）
    DELAY_US(3000L);                           // 等待外部晶体正常工作
    SysCtrlRegs.CLKCTL.bit.XCLKINOFF = 1;      // 关闭外部有源时钟输入
    SysCtrlRegs.CLKCTL.bit.OSCCLKSRC2SEL = 0; // 选择外部时钟源通路
    SysCtrlRegs.CLKCTL.bit.OSCCLKSRCSEL = 1;   // PLL输入时钟源由内部OSC1切换至外部时钟源
    SysCtrlRegs.CLKCTL.bit.WDCLKSRCSEL = 0;    // 看门狗选择内部OSC1
    SysCtrlRegs.CLKCTL.bit.INTOSC2OFF = 1;     // 关闭片内OSC2
    SysCtrlRegs.CLKCTL.bit.INTOSC1OFF = 0;     // 开启内部OSC1
    EDIS;
}
```

等待延时18ms

切换外部时钟

图 3-1

4. 时钟源配置流程图

在切换外部时钟前，由于用户软硬件差异化（如下图 3-1 所示），导致系统初始化时间不确定，所以建议用户在程序切时钟源前等待时间大于 15ms,以确保程序正常运行。

```
void main(void)
{
    DisableDog();
1   MemCopy(&RamfuncsLoadStart, &RamfuncsLoadEnd, &RamfuncsRunStart);
    InitSysCtrl();
    InitGpio();
    InitFlash();
    InitPieCtrl();
    InitPieVectTable();
    /*
    用户外设初始化
     */
    CallUserAPP;
}
```

```
void InitSysCtrl(void)
{
   DisableDog();
   EALLOW;
   SysCtrlRegs.PCLKCR0.bit.ADCENCLK = 1; // Enable ADC peripheral clock
   (*Device_cal)();
   SysCtrlRegs.PCLKCR0.bit.ADCENCLK = 0; // Return ADC clock to original state
   EDIS;
   XtalOscSel();
   InitPll(DSP28_PLLCR,DSP28_DIVSEL);
   InitPeripheralClocks();
}
```

```
void XtalOscSel (void)
{
   EALLOW;
   SysCtrlRegs.CLKCTL.bit.XTALOSCOFF = 0;     // 使能内部振动器（默认使能）
2  DELAY_US(3000L);                           // 等待外部晶体正常工作
   SysCtrlRegs.CLKCTL.bit.XCLKINOFF = 1;      // 关闭外部有源时钟输入
   SysCtrlRegs.CLKCTL.bit.OSCCLKSRC2SEL = 0; // 选择外部时钟源通路
   SysCtrlRegs.CLKCTL.bit.OSCCLKSRCSEL = 1;   // PLL输入时钟源由内部OSC1切换至外部时钟源
   SysCtrlRegs.CLKCTL.bit.WDCLKSRCSEL = 0;    // 看门狗选择内部OSC1
   SysCtrlRegs.CLKCTL.bit.INTOSC2OFF = 1;     // 关闭片内OSC2
   SysCtrlRegs.CLKCTL.bit.INTOSC1OFF = 0;     // 开启内部OSC1
   EDIS;
}
```

图 4-1

5. 时钟选择配置注意事项

1. 延时函数 DELAY_US（）定义如下所示，定义段为 ramfuncs，此段需搬移至 ram 运行，否则导致程序运行异常，进入非法陷阱中断，如上图 4-1 红色❶。

```
*****************************************************************************************
#define DELAY_US(A)  DSP28x_usDelay(((((long double) A * 1000.0L) / (long double)CPU_RATE) - 9.0L) / 5.0L)
    .def _DSP28x_usDelay
    .sect "ramfuncs"
    .global  __DSP28x_usDelay
_DSP28x_usDelay:
     SUB    ACC,#1
     BF     _DSP28x_usDelay,GEQ    ;; Loop if ACC >= 0
     LRETR
*****************************************************************************************
```

2. DELAY_US(A) 延时函数是基于主频 60MHZ 计算的，在时钟切换调用此函数时,PLL 还未完成初始化，此时主频为 10MHz(内部 OSC 提供)，所以在切换时钟前 DELAY_US(3000L) 延时约 18ms，如上图 4-1 红色❷。

6. 时钟切换异常导致影响

用户程序未按正确流程及时序初始化，可能导致如下影响：

- 程序进入非法陷阱中断，无法正确运行；

```
__interrupt void ILLEGAL_ISR(void)
{
      __asm("         ESTOP0");
      for(;;);
}
```

- 芯片出现时钟检测丢失导致局部复位，如果如下程序红色箭头未定义（默认），最终程序可能会正确运行（多次局部复位最终满足外部晶体正常工作时序要求）。

```
void InitPll(Uint16 val, Uint16 divsel)
{
   volatile Uint16 iVol;
   if (SysCtrlRegs.PLLSTS.bit.MCLKSTS != 0)
   {
      EALLOW;
      SysCtrlRegs.PLLSTS.bit.MCLKCLR = 1;
      EDIS;
      __asm("       ESTOP0");   <-- 此条语句仅在仿真起作用，脱机运行等效空操作
   }
   if (SysCtrlRegs.PLLSTS.bit.DIVSEL != 0)
   {
       EALLOW;
       SysCtrlRegs.PLLSTS.bit.DIVSEL = 0;
       EDIS;
   }
   if (SysCtrlRegs.PLLCR.bit.DIV != val)
   {
      EALLOW;
      SysCtrlRegs.PLLSTS.bit.MCLKOFF = 1;
      SysCtrlRegs.PLLCR.bit.DIV = val;
      EDIS;
      DisableDog();
      while(SysCtrlRegs.PLLSTS.bit.PLLLOCKS != 1);

      EALLOW;
      SysCtrlRegs.PLLSTS.bit.MCLKOFF = 0;
      EDIS;
    }

    if((divsel == 1)||(divsel == 2))
    {
        EALLOW;
        SysCtrlRegs.PLLSTS.bit.DIVSEL = divsel;
        EDIS;
    }
    if(divsel == 3)
    {
        EALLOW;
        SysCtrlRegs.PLLSTS.bit.DIVSEL = 2;
        DELAY_US(50L);
        SysCtrlRegs.PLLSTS.bit.DIVSEL = 3;
        EDIS;
    }
}
```